Part VI

Version Control

Why version control?

- When writing software, you may imagine it in terms of continual improvement.
- More likely, you will usually be making improvements, but you may want to experiment with new features, completely refactor your code, or similar.
- Or, you may manage to introduce a bug, and fail to notice it for a few weeks...
- In all of these cases, it would be helpful to roll-back your code (or parts of it) to an earlier version.
- One way to do this would be to save a copy of your program every day with a date-related filename. This way lies `EulerSolver_WORKING_1D_050426_b` and worse.

What version control provides

- Version control generally allows you to save changes made to particular files with an associated message.
- You can interrogate the system to extract an old version of the code into a separate directory.
- You can roll-back just the changes to one file without touching others.
- You can share your code with other people and ensure that you all know what your different code versions are based on.

Version control models

There are two main models for version-control:
Client-server model:

- The repository is (usually) stored an a remote server.
- All users have appropriate access to this server to extract code or to commit their changes.
- This is suited to a company model, where there is an official version of the project, centrally stored.
- Commits, and some history queries will require network access to this server.
- In some cases, the client-machine can double-up as the server.

Distributed model

Distributed model:

- The entire repository is stored on each user's client.
- If developers need to share the repository, they can either set up a central server with an official repository version, or give other developers read-access to their repository, so that they can pull changes from that repository.
- Some systems with this model (e.g. git) allow multiple repositories for one project, so you can have a local repository for your personal changes, and a centralized one for collaboration.

Version control systems

- The most popular ones in use at the moment are git and Subversion
- Other options are available, such as CVS and Mercurial, as well as other less-used open-source options, and some proprietary ones.
- Subversion was developed in the early 2000s by CollabNet, a company started by Tim O'Reilly (of O'Reilly books)
- git was developed by Linus Torvalds to aid development of the Linux kernel around 2005.

Which one should I use?

- If you are working on an existing project, then the decision will have been made for you already.
- If this is a major project (commercial or academic) that does not use version control, back away slowly...
- These days, `git` is the most common approach in my experience. Older code-bases may still be on other platforms.

Git hosting

- Although Git can just be used locally, you usually have a remote repository that you sync. to.
- Many of you may have GitHub accounts already.
- Pros: It's free, widely available.
- Cons: Run by Microsoft.
- If you use it for MPhil. projects, make sure your repository is Private; otherwise other people can re-use your code without your knowledge.
- Alternatives include the University's GitLab:
  `https://gitlab.developers.cam.ac.uk/`
  which uses your University login.

Git

Getting started:

- Either you will be working on an existing repository, in which case:

```
git clone https://github/pmblakely-coding/trunk/MainCode
```

- or you will need to create your own repository:

```
git init /home/pmb39/WorkingCode
```

- All repository history is stored in special `.git` directories.
- You should ensure that the files are all backed up (or at least sync'ed to a remote repository).
- In order to update to the latest version of the code from a remote repository, use:

```
git pull
```

Basic usage

- You need to indicate that certain files are to be added to your repository:

```
git add ./Driver.C ./IdealGas.C ./IdealGas.H settingsFiles/
```

- At this point, the files are not committed to the repository, just marked as being files to be eventually put into the repository.
- Addition of directories will happen recursively, and may add more than you intended.
- To add only files that are already tracked by git, use:

```
git add -u settingsFiles/
```

  which will mark any updated files in `settingsFiles/` for committing to the repository.

Deleting files

- If you need to delete files from your code, then:

```
git rm ./IdealGas.C
```

- You may wish to keep the file locally on your file-system, but only delete it from the head of the repository:

```
git rm --cached ./IdealGas.C
```

- Remember that if a file has been committed to the repository then it remains there in perpetuity in earlier revisions, and you can access it by checking-out the appropriate revision.
- (Deleting a file's complete history is a tricky operation - intentionally so.)

Initial commit

- In order to commit the newly added files to the repository:

```
git commit
```

- A text-editor (`nano` by default) will open and ask for a commit message.
- Type something useful then exit the editor.
- Any additions/changes to your files will be transferred to the repository.
- If you now type `git log` you will see the initial commit.

Other operations

- Other operations are possible:
  `git mv IdealGas.C StiffenedGas.C`
  will move one file to the new place, and take account of the file's history.
- If you do:
  `cp IdealGas.C StiffenedGas.C`
  `git add StiffenedGas.C`
  then `git` will (very cleverly) work out that the second file shares history with the first.
- Having this link can be useful for checking who wrote a line of code even if the code has been copied and/or moved.

Local changes

- While developing your code, you may need to check what changes you have made recently, compared to the checked-out version.

```
> git status .
# Changes to be committed:
# (use "git reset HEAD <file>..." to unstage)
#       new file:   StiffenedGas.C
# Changes not staged for commit:
# (use "git add <file>..." to update what will be committed)
# (use "git checkout -- <file>..." to discard changes in
    working directory)
#       modified:   IdealGas.C
#
# Untracked files:
#   (use "git add <file>..." to include in what will be
    committed)
#       MieGruneisen.C
```

- This shows that `IdealGas.C` has local changes (but not staged for commit), `StiffenedGas.C` has been staged for addition (but not committed), and `MieGruneisen.C` has not been staged for addition.

Local changes ctd - git

- While developing your code, you may need to check what changes you have made recently.

```
> git diff ./IdealGas.C
diff --git a/IdealGas.C b/IdealGas.C
index 2d7a527..83ea3fc 100644
--- a/IdealGas.C
+++ b/IdealGas.C
@@ -73,5 +73,5 @@ git commit
    const double v_y = state[V_Y];
    // Calculate kinetic energy
-   const double ke = rho*(v_x*v_x + v_y*v_y);
+   const double ke = 0.5*rho*(v_x*v_x + v_y*v_y);
    const double totalEnergy = ke + internalEnergy;
```

  which shows the differences between your local version and the staged version of the file.

- This is not (necessarily) the same as the absolute latest version of the repository if someone has pushed a later change to the central repository that you have not `pull`ed.

Roll-back to latest version

- You may decide that all the changes you have made since your last commit or update are useless.
- In this case, you can roll back your working copy to be the same as the latest commit:
  `git checkout ./IdealGas.C`
- which completely discards the local changes, and there is no way to recover them.

Further committing - git

- Once you have made various changes to your files, you tell git that you want a particular set of files to be committed to the repository:

  ```
  git add IdealGas.C
  ```

  and then commit the changes

  ```
  git commit IdealGas.C
  ```

- A text-editor will open; if you don't like `nano` you can use:

  ```
  git config --global core.editor emacs
  ```

  or set the environment variable `SVN_EDITOR=vi`

- You should enter a useful log-message, e.g. "Corrected kinetic-energy calculation".
- Make your commit messages useful; future-you will thank you.
- Save the file, and exit the editor.

Notes on git's staging

- You will note that git uses a two-stage process when committing changes.
- In git, you have to explicitly schedule modified files to be committed to the repository.
- This allows you to break commits down into a set of logically separate commits, and forces you to think about each modified file.
- git therefore forces clearer thinking by default, as compared to Subversion, for example.

When to commit

- You could `git commit` at the end of each day.
- This will sort-of work.
- Ideally, you make sure that each committed-state of the repository corresponds to a plausibly working version of the code.
- Do not mix unrelated changes to various files.
- Each commit should be a self-contained single set of changes.
- For example, fixing a specific bug, changing the way a particular function is called, etc.
- This makes future history-examining easier.

What should be in the repository?

- The repository should contain everything required to recompile and run your program(s) to obtain verifiable data.
- Source files (`.f .F .c .C .h .H` etc.), Makefiles, settings files, post-processing scripts, etc.
- It should not contain files that can be generated from these, e.g. object files, executables, PDFs of papers that are needed as documentation, and other output files.
- Most version-control systems work best with pure-text-based files, as differences are easy to determine with `diff`.
- Binary (non-text) files can sometimes go into repositories, but you are unlikely to get anything useful except individual revisions of the files as they are probably in a complex/compressed format.
- Pictures for inclusion in documentation are one type of binary file that might go in a repository.

Ignoring certain files

- In order to avoid accidentally putting compiled files into your repository, you can create a `.gitignore` file:

```
# Ignore object files
*.o
# Ignore emacs temp. files
*~
# Ignore log-files
*.log
```

- This will cause Git always to ignore files with the given file-pattern and never add them by default.

Querying the repository

`git log ./IdealGas.C`
displays the commit messages associated with changes to the named file.

```
commit d6f39d56a5179d150437827abf7292f91827cbe2
Author: Philip Blakely <pmb39@cam.ac.uk>
Date:   Tue Aug 5 11:55:43 2025 +0100
    Add Ideal-Euler and Euler-Ideal mixed Riemann solvers.
    Required addition of extra functions to EoS for solving
    Riemann
    problems.
```

Here we see the commit-id (useful for checking-out earlier code versions).
Using `git log` alone gives the history of the entire repository.

Working with the repository

- If you find that your code has stopped working, you can go back to previous revisions.
- Firstly, if you have any local changes to files, you should save a separate copy of them to avoid any loss of work. (Either `git commit` or `git stash`.)
- Then, roll back to an earlier revision:

  ```
  git checkout 1c1f9c2dd44044ba1ab777be89fe08ae4e351cff
  ```

  where the commit-id comes from reading `git log`
- (You can use a shorter, unique, commit-id as well; 6-8 characters are usually sufficient.)
- You can compile and test your program again and see if the same error was in that code version.

Working with the repository

- To see what changes were made between the current version and an older version:

```
git diff 1c1f9c2d
```

- will give all changes to all files between the current and older commit.
- You can also find the differences between two versions

```
git diff 1c1f9cd..d6f39d56
```

- To finally go back to the latest code version:

```
git checkout main
```

Branching

- There are at least two reasons to use branches in version control.
- Firstly, you may want to develop a complex feature into your code, which will take a lot of work (i.e. many commits) to get right. A branch will allow separate development of this feature while your original version can still be used, and modified independently.
- Secondly, you may want to preserve an older version of your code, and maintain it as a bug-fix version, i.e. you can make minor corrections to it, even though you are making large modifications to the main code.
- You will notice that these are more-or-less the same thing viewed from different angles.

Branching in git

- To create a branch of your code in git:

```
git branch Refactor
git switch Refactor
```

- Now, the original **trunk** code version still points to the original code version, but so does **Refactor**.
- Any further commits will happen on the **Refactor** branch instead.
- To switch back to the original branch:

```
git switch main
```

  (or possibly **development** instead of **main**).

- You can merge changes from **main** into your current branch with:

```
git merge main .
```

- (Merging can get messy; beyond the scope of this lecture.)

Bisection

Suppose you discover that your code fails to solve a standard problem correctly, even though it did so three months (and 100 commits) ago.

- Write a simple script that compiles your code and runs it against the standard problem and checks it against the correct solution.
- The script should exit with code 0 if the correct solution is found, otherwise with code 1 (or 125 to skip - e.g. if the code does not compile).
- Then, in a clean, checked-out version of your repository:

```
git bisect start
git bisect good a7398ffdb32
git bisect bad 9e51c89b32f
git bisect run ./test.sh
```

- This will use a bisection method to find the first revision where the test-case breaks.

Commit separation

- What happens if you are working on two separate changes to the code at once, or notice a simple mistake while working on a major rewrite?
- Here, you should commit only the changes to one file, while keeping the rest as a local modification.
- If you have two sets of changes to one file that are somehow independent, then you will need to take a copy of the file, revert the original, and make only one set of changes before committing again, and then making the remaining set of changes.
- If this happens a lot, you may need to reconsider how well your project is divided into separate files.
- `git` only works with files at its most granular level.
- (However, see `git add -p` which allows you to pick which parts of files to commit.)

VCS as documentation

- One day you may find yourself wondering why a particular line is what it is.
- Of course, this should be obvious, given that your code is well-documented...
- If not, though, you could run:

```
git blame EulerEquations.C
   b243f11e pmb39   for(int d=0 ; d < 3 ; d++) {
   b243f11e pmb39     const real V_d = prim[VEL+d];
   c1ed290f stm31     cons[MOM + d] = rho * V_d;
   b243f11e pmb39     vSquared += V_d * V_d;
   b243f11e pmb39   }
```

  which will list the file with the last revision at which each line was changed (here Steve fixed my momentum calculation).
- You can then look up the commit message associated with that commit, which may also provide some help. This is not really a substitute for good documentation, however.

Backups

- As ever, backups are vital.
- Use the `git bundle` command or ensure that the directory containing your repository is backed up.
- If you lose the repository, you will have lost the development history of your code, even if your code is intact.
- `git` can also push commits to a remote repository, e.g. GitHub (owned by Microsoft) or GitLab (run by GitLab Inc.)
- Cambridge has its own GitLab installation: `https://gitlab.developers.cam.ac.uk/`

GUIs

- In some cases, it is useful to see an overview of the repository and its history in a more visual form.
- git comes with its own GUIs: git-gui and gitk
  - `git gui` geared towards staging files for commit
  - `gitk` geared towards examining repository history

Summary

- This should have given you some idea of the features of git.
- It is worth trying them on a small or dummy project first to get the hang of them before using them on a major project.

References

The main reference to `git` is freely available in PDF and web-based format and is very well written with various examples and diagrams.

- `https://git-scm.com/doc`
- `https://www.atlassian.com/git`

Course feedback

Please help me improve this course for future students by providing feedback:

